



الأكاديمية العربية للعلوم والتكنولوجيا والنقل البحري

Arab Academy for Science, Technology & Maritime Transport

Course: Mobile Application

Course Code: ECE5605

Dr. Ahmed Maher

Eng. Heba Kotb

<i>Name</i>	<i>ID</i>
Mohamed Tarek	232004026
Abdelrahman Ayman	221005844
Salaheldin Mostafa	221004454
Marwan Ahmed Salah	221006690
Ali Essam	221005093
Osama Mohamed	221006341

Medicine Reminder App
(MediCare)

Table of Contents

1. Introduction.....	3
1.1 Purpose.....	3
1.2 Scope.....	3
1.3 Definitions, Acronyms, and Abbreviations.....	3
1.4 Overview.....	4
2. Overall Description.....	5
2.1- Product Perspective	5
2.2- Product Functions	5
2.3- User Classes and Characteristics	6
2.4- Operating Environment	7
2.5- Design and Implementation Constraints.....	7
2.6- Assumptions and Dependencies	7
3. Functional Requirements	8
4. Non-Functional Requirements	14
5. Use Cases	15
6. System Design (with diagrams + UX <i>Figma</i>)	17
7. Schedule and Budget.....	27
7.1 Project Schedule.....	27
7.2 Budget.....	27
8. Appendices.....	27
References.....	28

1. Introduction

[Salaheldin Mostafa]

1.1 Purpose

This document presents the Software Requirements Specification (SRS) for the MediCare application, a mobile-based medicine reminder system. The purpose of this report is to define the functional and non-functional requirements of the system, as well as its overall design and behavior.

1.2 Scope

MediCare is a mobile application designed to help users to manage their daily medication schedules. The system provides features such as medication reminders, dose tracking, history monitoring, adherence statistics, and onboarding, to improve adherence and reduce missed doses. Users can add medicines with custom types, dosage, repeat-day schedules, and color coding. The app sends local notifications, tracks status (Taken / Taken Late / Pending / Missed), displays a 7-day history, and shows adherence statistics. All data is stored locally on the device using shared preferences.

The application allows users to add medicines, receive notifications at specific times, and track whether medications are taken, pending, or missed. It aims to provide a simple, reliable, and user-friendly solution suitable for a wide range of users.

1.3 Definitions, Acronyms, and Abbreviations

- **App:** Mobile application (MediCare)
- **Local database:** Backend platform used for authentication and database
- **Notification:** Reminder alert sent to the user at a scheduled time
- **UUID:** Universally Unique Identifier used for generating medicine IDs
- **SRS:** Software Requirements Specification

1.4 Overview

Medication consistency is a critical factor in maintaining good health, especially for individuals who rely on daily medications to manage chronic or ongoing conditions. However, many people struggle to remember taking their medicine on time because of their busy schedules, lack of organization, or simply forgetting. This can lead to missed doses, reduced treatment effectiveness, and probably health risks.

With the improvements of mobile technology, smartphones have become an effective tool for managing daily activities. Mobile applications provide an accessible way to remind users, track habits, and improve consistency.

The MediCare application is developed using Flutter and local storage (`shared_preferences`) to provide a reliable, offline-capable solution for medication management.

The application focuses on simplicity, usability, and reliability, showing how new mobile technologies can be used to address everyday health-related challenges effectively.

2. Overall Description

[Mohamed Tarek]

2.1- Product Perspective

A lot of people face difficulties in maintaining consistent medication schedules, especially when they are required to take more than one medicine at different times throughout the day.

Forgetting to take medication is a common issue caused by busy daily routines, lack of reminders, or poor time management. This problem is more significant for patients with chronic illnesses, old age individuals, and even students or working professionals who may forget their medication due to distractions.

Old methods such as written notes, alarms, or relying on memory are often unreliable and insufficient for effective medication management. For example, alarms do not give context about which medicine to take and do not track whether the medication was taken or not. This can lead to confusion and forgetting.

Additionally, there is a lack of simple and centralized systems that allow users to track their medication history. Without proper tracking, users cannot easily review their adherence or identify patterns of missed doses, which may negatively affect treatment outcomes.

Therefore, there is a need for a simple, intuitive, and efficient mobile application that helps users manage their medications, provides timely reminders, tracks daily doses, and presents clear status indicators without unnecessary complexity.

2.2- Product Functions

The Medicine Reminder Application provides the following functions:

- User Registration and Authentication
 - Users can create accounts, log in securely, change passwords, and log out.
- Medication Management
 - Users can add, edit, and delete medications.
 - Each medication includes a name, dose, type, reminder time, repeat days, and optional notes.
- Reminder Notifications
 - The application sends local notifications at scheduled medication times.
 - Notifications continue to work even when the application is closed.

- Medication Tracking
 - Users can mark medications as taken, pending, or missed.
 - The system records medication adherence history.
- History Management
 - Users can review their medication history and adherence records.
- Statistics Dashboard
 - Users can view adherence statistics, medication streaks, taken doses, missed doses, and weekly performance charts.
- Profile Management
 - Users can edit profile information, change passwords, and log out.
- Offline Operation
 - The application stores data locally and does not require an internet connection for normal operation.

2.3- User Classes and Characteristics

The MediCare application is designed for a wide range of users who need assistance in managing their medication schedules.

The primary users include patients with chronic illnesses such as diabetes or hypertension, who require regular and timely medication intake. These users need a reliable system to ensure adherence.

Elderly individuals are also a key user group, as they may experience memory challenges and require a simple and intuitive interface.

Students and working professionals represent another user group, as their busy schedules may lead to missed doses. The application helps them stay organized through reminders and tracking.

Caregivers and family members can also use the application to help monitor and manage medications for others.

All users are expected to have basic smartphone usage skills.

2.4- Operating Environment

The MediCare application operates on mobile platforms, primarily Android devices, and can be extended to iOS due to the use of Flutter.

The system runs on a smartphone and uses a local database for storing user and medicine data. Internet connectivity is not required for core functionality, as all operations are performed locally on the device.

2.5- Design and Implementation Constraints

Programming language: Flutter (Dart), targeting Android and iOS.

Local storage: `shared_preferences` is the sole persistence layer. No Firebase or cloud database is used.

Data is stored per user using base64-url-encoded email as a key namespace to prevent data collision between accounts.

A schema version number is stored; if the app detects an upgrade, it clears stale data to ensure integrity.

Project developed within a limited academic timeframe.

2.6- Assumptions and Dependencies

It is assumed that users have access to a smartphone capable of running the application.

The system depends on the proper functioning of the local storage system on the device.

It is also assumed that users will allow notification permissions to receive reminders.

Users grant exact-alarm permissions on Android 12+ for precise notification scheduling.

The device clock is reasonably accurate for dose status calculation.

3. Functional Requirements

[Osama Mohamed]

The functional requirements define the core features and operations provided by the MediCare application. These requirements describe how the system behaves in response to user inputs and interactions.

3.1 Feature: User Registration

- **Description:**
Allows new users to create an account using email and password.
- **Inputs:**
Email, Password
- **Process:**
The system stores user credentials securely in the local database.
- **Output:**
User account is successfully created and stored securely.

3.2 Feature: User Login

- **Description:**
Allows existing users to log into their accounts.
- **Inputs:**
Email, Password
- **Process:**
The system verifies user credentials using locally stored data.
- **Output:**
Access granted to the application or an error message if credentials are invalid.

3.3 Feature: Add Medicine

Description:

Allows users to create a new medication reminder with detailed scheduling information.

Inputs:

- Medicine name
- Dose amount
- Medicine type (Tablet, Capsule, Liquid, Injection, Inhaler, Drops)
- Reminder time
- Repeat days
- Optional notes

Processing:

- The system validates all required information.
- The medication is stored in local storage.
- A local notification schedule is created for the selected days and times.

Outputs:

- Medication is successfully added.
- Reminder notifications are scheduled.

Precondition:

- User must be logged in.

Postcondition:

- The medication appears in the medication list and scheduled reminders become active.

3.4 Feature: View Medicines (Home Screen)

- **Description:**
Displays all medicines associated with the user along with their daily status.
- **Inputs:**
User ID
- **Process:**
The system retrieves medicine data from the local database and calculates the status (Taken, Pending, Missed).
- **Output:**
List of medicines with their scheduled time and status.

3.5 Feature: Mark Medicine as Taken

- **Description:**
Allows users to confirm that a medicine dose has been taken.
- **Inputs:**
Medicine ID, Current date
- **Process:**
The system creates a taken record stored in the local database
- **Output:**
Medicine status is updated to “Taken” and reflected in the UI.

3.6 Feature: Notifications

- **Description:**
Three notifications are sent per medicine per day: (1) a pre-dose alert before the window opens, (2) an on-time reminder at the scheduled time, and (3) a missed-dose alert after the overdue threshold passes. Uses `zonedSchedule` with exact alarm or inexact fallback.
- **Inputs:**
Medicine time
- **Process:**
The system schedules local notifications using the `flutter_local_notifications` package. Notifications are scheduled using local device services.
- **Output:**
Notification is displayed at the specified time, even when the app is closed.

3.7 Feature: Notification Interaction

- **Description:**
Handles user interaction with notifications.
- **Inputs:**
Notification tap
- **Process:**
The system opens the application and navigates to the Home screen.
- **Output:**
User is redirected to the relevant screen for quick action.

3.8 Feature: View History

- **Description:**
Displays the user's medication adherence over the past seven days.
- **Inputs:**
User ID
- **Process:**
The system retrieves data from the local database.
- **Output:**
A list or table showing taken/missed status for each medicine.

3.9 Feature: Profile Management

- **Description:**
Allows users to manage their account.
- **Inputs:**
 - Update Name
 - Update phone Numbers
 - Current password
 - New password
- **Process:**
 - The system validates profile changes.
 - The system updates stored user information.
 - The system verifies credentials before changing passwords.
- **Output:**
 - Profile information updated successfully.
 - Password changed successfully.
 - User can log out securely.

3.10 Feature: Statistics Dashboard

- **Description:**
Allows users to monitor their medication adherence through visual statistics and performance summaries. Allows users to monitor their medication adherence through visual statistics and performance summaries.
- **Inputs:**
 - Medication history records
 - Medication status data (Taken, Missed, Pending)
- **Process:**
 - The system retrieves medication history data.
 - The system calculates adherence percentages.
 - The system calculates taken and missed doses.
 - The system generates weekly performance statistics.
 - The system calculates medication streaks.
- **Output:**
 - Adherence percentage
 - Total taken doses
 - Total missed doses
 - Weekly medication statistics
 - Medication streak information

3.11 Feature: Navigation

- **Description:**
Enables movement between application screens.
- **Inputs:**
User interaction (buttons, taps)
- **Process:**
The system uses Flutter navigation to switch between screens.
- **Output:**
User is redirected to the selected screen.

3.12 Feature: Status Calculation

- **Description:**
Determines the daily status of each medicine.
- **Inputs:**
Medicine time, current time, taken records
- **Process:**
The system compares the current time with the scheduled time and checks if a record taken exists.
- **Output:**
Status is displayed as Taken, Pending, or Missed.

4. Non-Functional Requirements

[Abdelrahman Ayman]

4.1 Performance

The system should respond to user interactions within 2 seconds under normal operating conditions.

The application should load the Home screen and display stored data within 3 seconds.

4.2 Security

User data must be stored securely within the local database on the device.

Access to the application should require user authentication using stored credentials.

The system should prevent unauthorized access to user data from other users on the same device.

Passwords are hashed before storage using SHA-256 (crypto package); plain-text passwords are never saved.

A schema version mechanism wipes stored data on upgrades to prevent stale or insecure data from persisting.

4.3 Usability

The application interface should be simple, intuitive, and easy to use for users with basic smartphone experience.

All main features (Add Medicine, Take Button, History) should be accessible within 2–3 interactions (clicks/taps).

4.4 Reliability

The system should operate without crashes during normal usage.

All user actions (e.g., adding medicine, marking as taken) must be saved correctly in the local database.

4.5 Availability

The application should be available for use at any time without requiring internet connectivity.

All core features must function offline

5. Use Cases

[Mohamed Tarek]

Use Case 1: User Registration

- **Actor:** User
- **Description:** User creates a new account in the system
- **Precondition:** User is not registered
- **Main Flow:**
 1. User opens the application
 2. User selects “Register”
 3. User enters email and password
 4. User submits the form
 5. System stores user data locally
- **Postcondition:** User account is created successfully

Use Case 2: User Login

- **Actor:** User
- **Description:** User logs into the application
- **Precondition:** User has an existing account
- **Main Flow:**
 1. User opens the application
 2. User enters email and password
 3. System verifies credentials
 4. User is redirected to Home screen
- **Postcondition:** User is logged in

Use Case 3: Add Medicine

- **Actor:** User
- **Description:** User adds a new medicine
- **Main Flow:**
 1. User opens the Add Medicine screen.
 2. User enters the medicine name.
 3. User enters the dose amount.
 4. User selects the medicine type.
 5. User selects the reminder time.
 6. User selects repeated days.
 7. User presses Save.
 8. System validates the information.
 9. System stores the medication.
 10. System schedules reminder notifications.
- **Postcondition:** Medicine appears in Home screen

Use Case 4: Mark Medicine as Taken

- **Actor:** User
- **Description:** User confirms taking medicine
- **Precondition:** Medicine is in Pending state
- **Main Flow:**
 1. User opens Home screen
 2. User clicks “Take” button
 3. System records action
 4. Status updates to “Taken”
- **Postcondition:** Medicine is marked as taken

Use Case 5: Receive Notification

- **Actor:** System
- **Description:** System reminds user to take medicine
- **Precondition:** Medicine time is scheduled
- **Main Flow:**
 1. System reaches scheduled time
 2. Notification is triggered
 3. User receives alert
 4. User taps notification
 5. App opens Home screen
- **Postcondition:** User is notified on time

Use Case 6: View History

- **Actor:** User
- **Description:** User views medication history
- **Precondition:** User is logged in
- **Main Flow:**
 1. User opens History screen
 2. System retrieves stored records
 3. System displays last 7 days data
- **Postcondition:** User reviews adherence

Use Case 7: View Statistics

Primary Actor: User

Description: The user views medication adherence statistics and performance reports.

Main Flow:

1. User opens the Statistics screen.
2. System retrieves medication history records.
3. System calculates adherence metrics.
4. System displays charts and summaries.
5. User reviews medication performance.

Alternative Flow:

- If no history data exists, the system displays a message indicating that statistics are unavailable.

6. System Design (with diagrams + UX *Figma*)

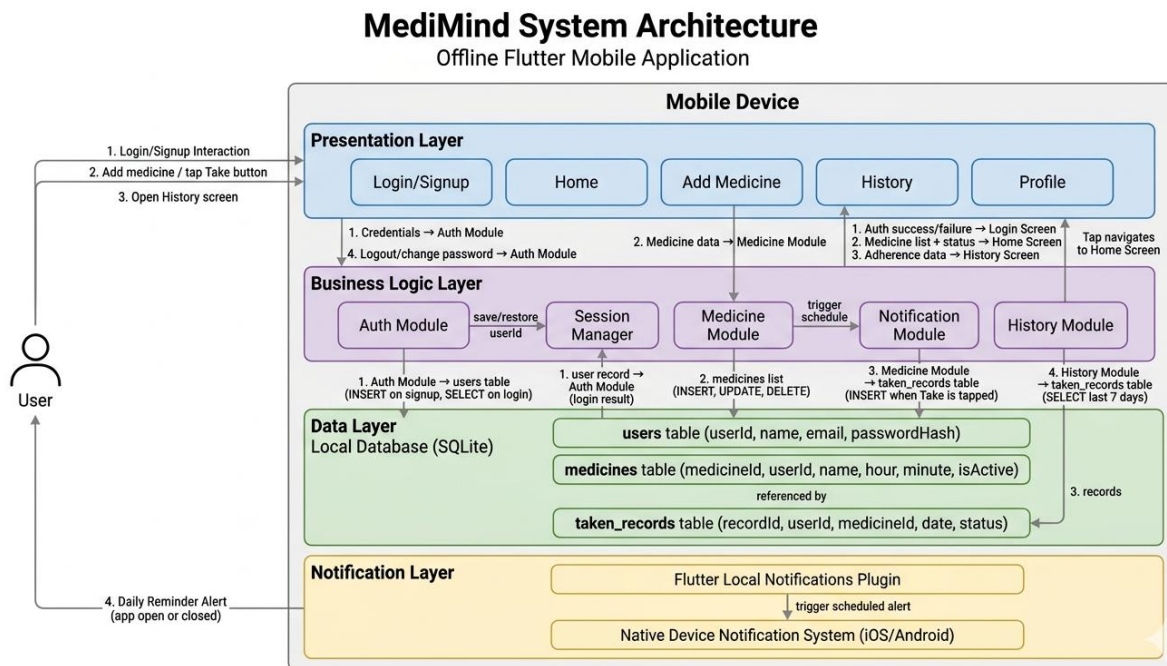
[Marwan Ahmed & Ali Essam]

6.1 System Architecture

The MediCare application follows a mobile-based architecture where the application handles both the user interface and data processing.

The system is developed using Flutter and uses a local database to store user and medicine data directly on the device.

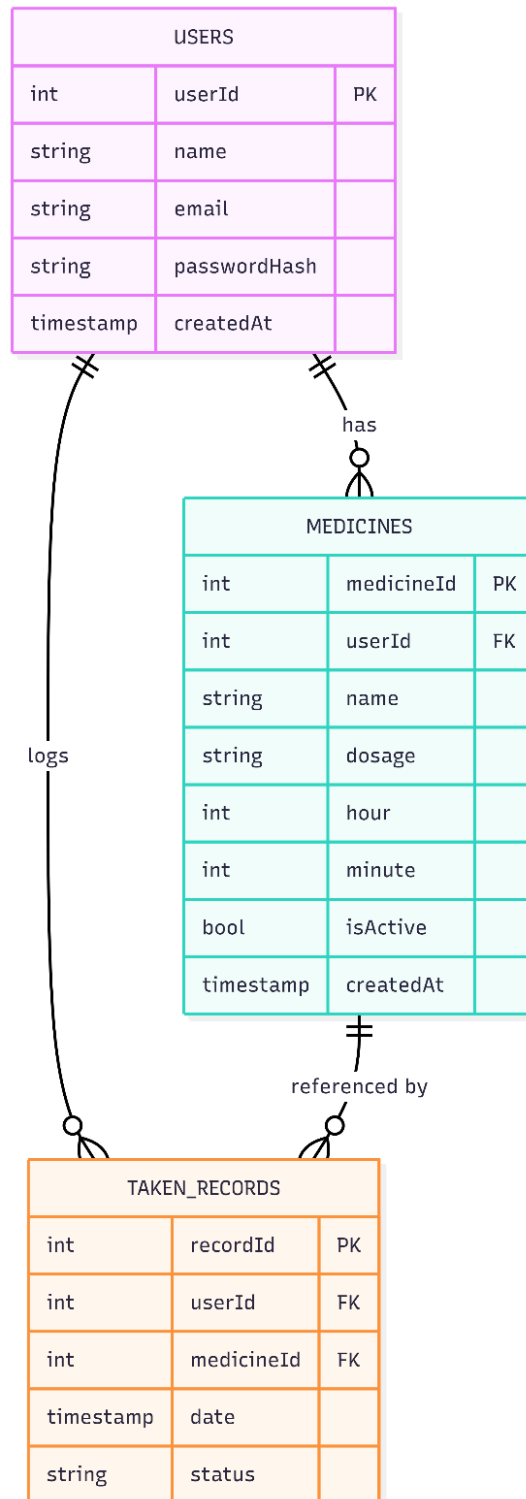
Local notifications are used to remind users of scheduled medications.



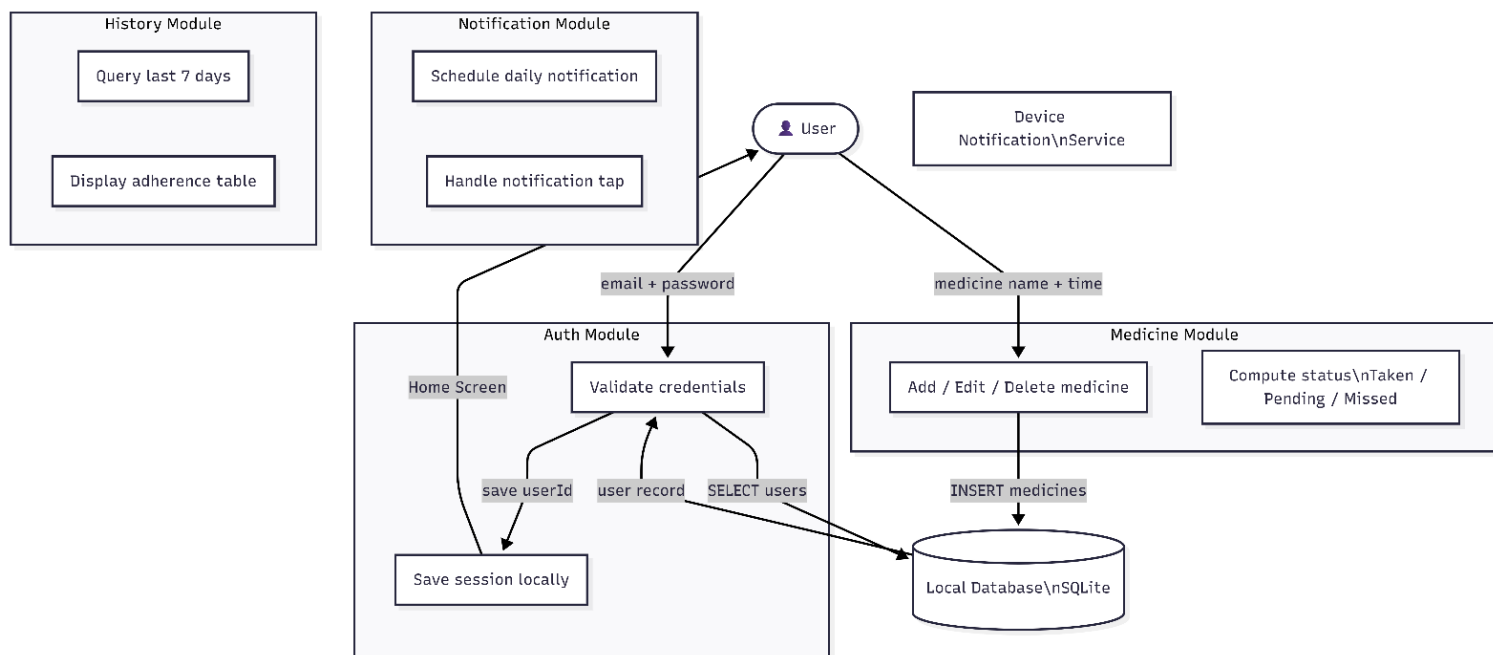
(Figure 1: System Architecture Diagram)

6.2 Data Design

This section presents the data-related design of the system.



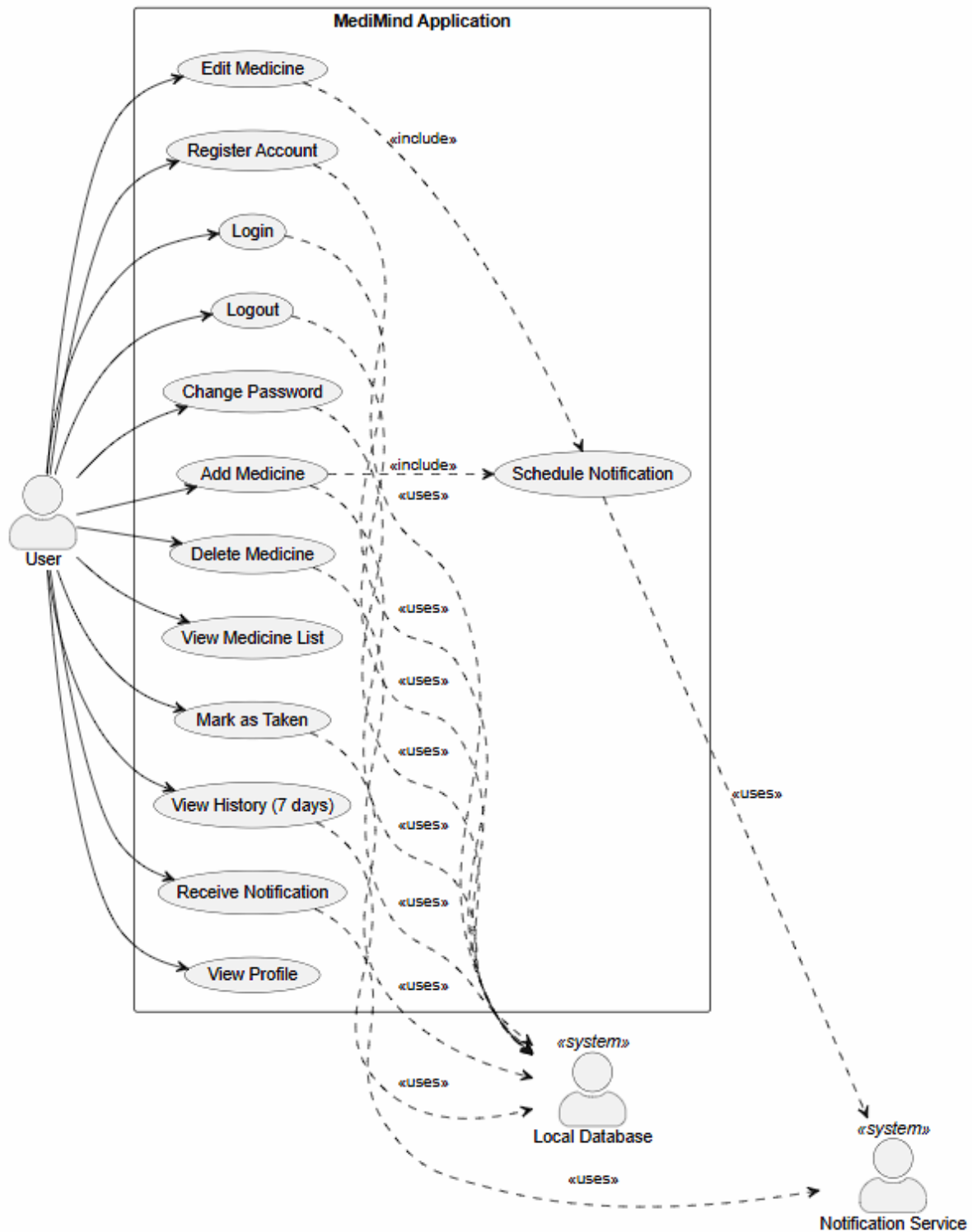
(Figure 2: ERD Diagram)



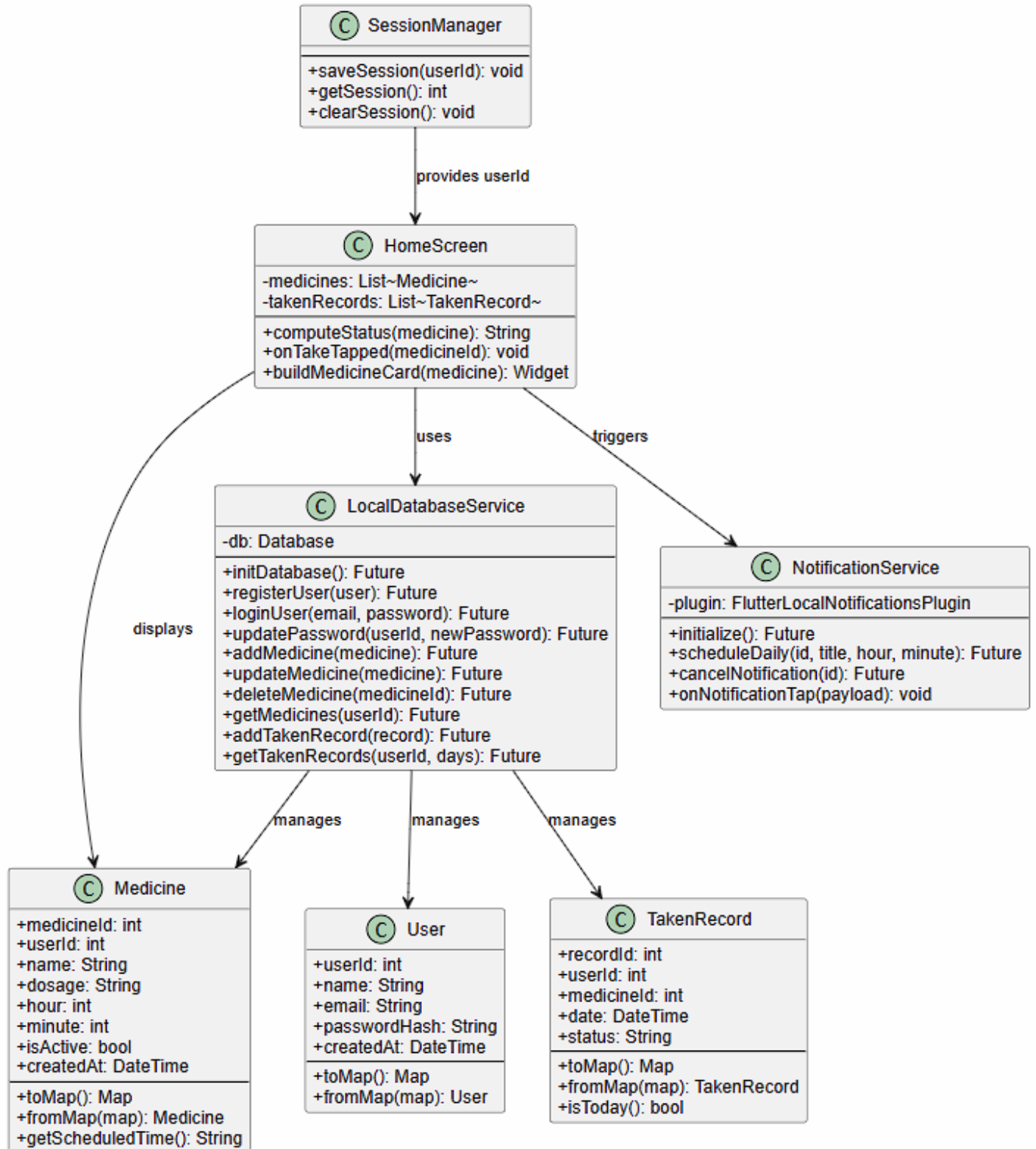
(Figure 3: Data Flow Diagram)

6.3 UML Diagrams

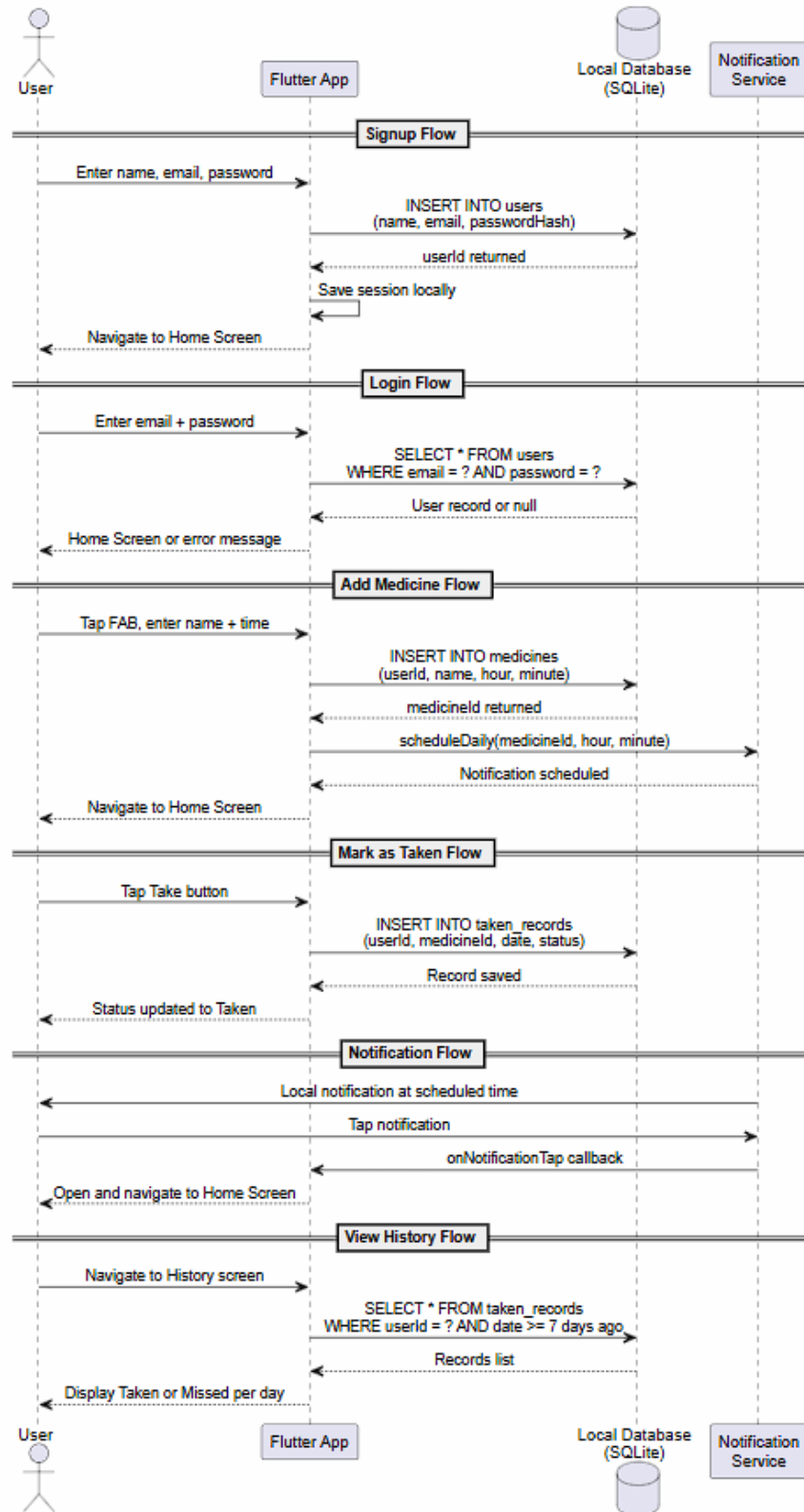
This section includes UML diagrams that describe system behavior and interactions.



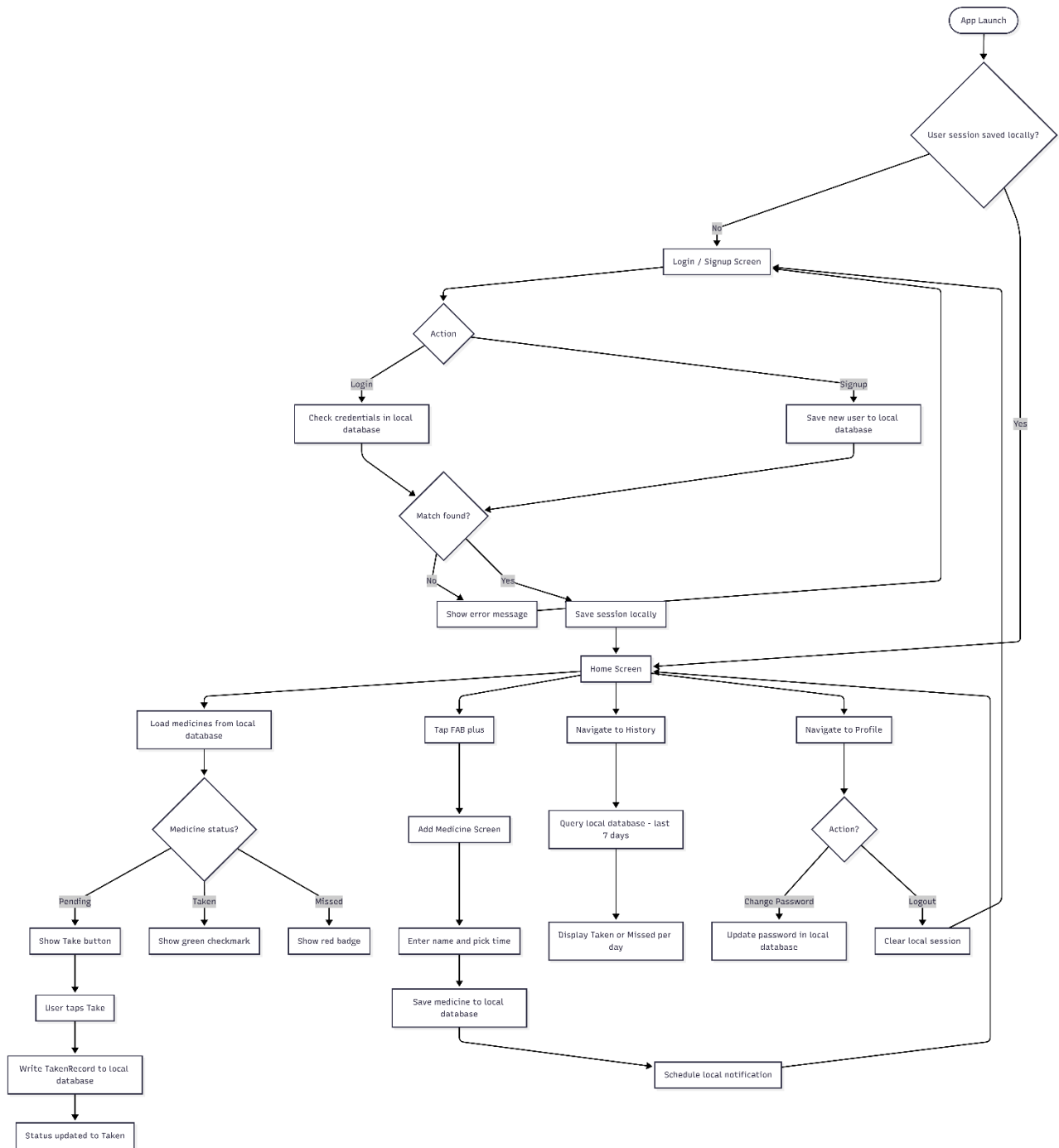
(Figure 4: Use Case Diagram)



(Figure 5: Class Diagram)



(Figure 6: Sequence Diagram)



(Figure 7: Flowchart)

All diagrams collectively describe the structure, data flow, and interaction within the system, providing a clear understanding of how the application operates.

6.4 User Interface (UX/UI Design)

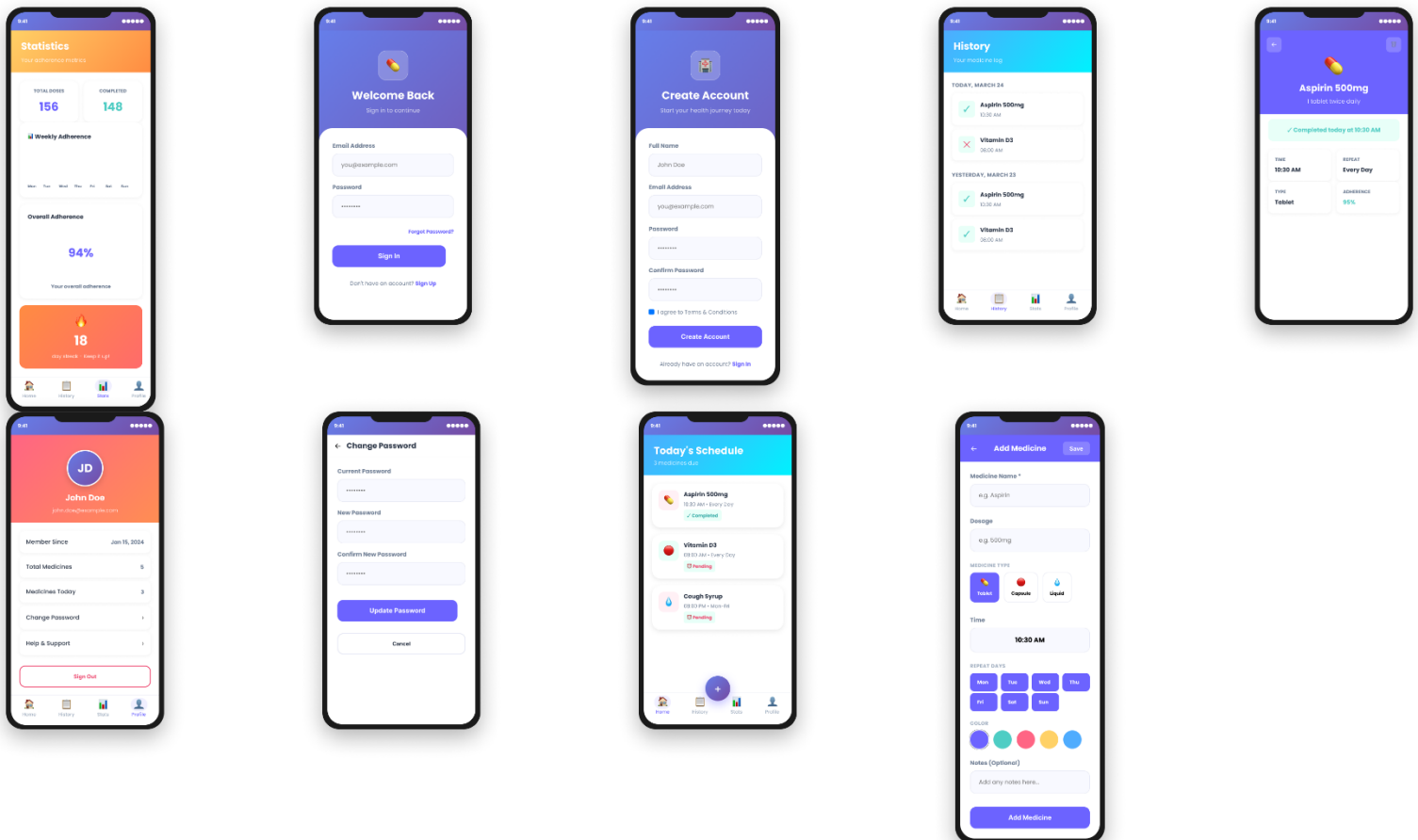
This section presents the user interface design of the MediCare application.

The UI/UX design was created using Figma to ensure a simple, clean, and user-friendly experience. (first draft)

The user interface design for the MediCare application was created using Figma. The full interactive design can be accessed at the following link: [Figma Design](#)

Screens include:

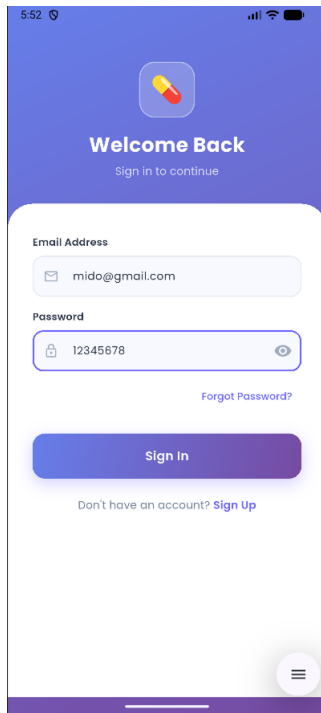
- Login and Registration screens
- Home screen (medicine list)
- Add Medicine screen
- Successful added medicine
- History screen
- Profile screen



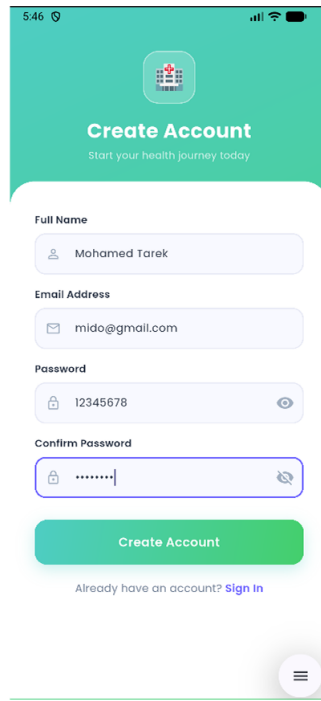
(Figure 8: UI Design – Figma Screens)

The design focuses on usability, clear navigation, and minimal user effort to perform tasks.

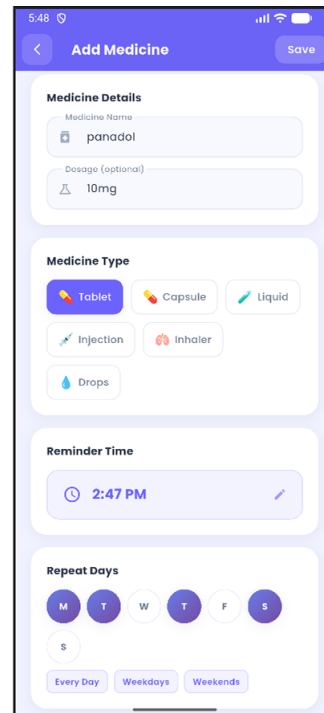
6.5 Real App User Interface



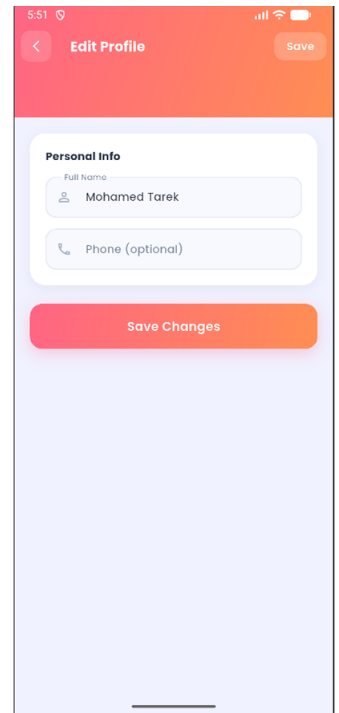
Sign In Screen



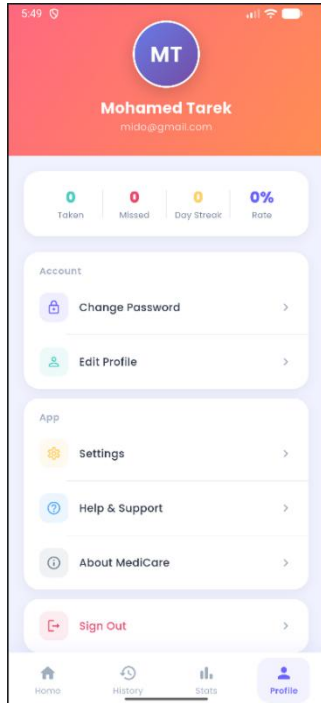
Sign Up Screen



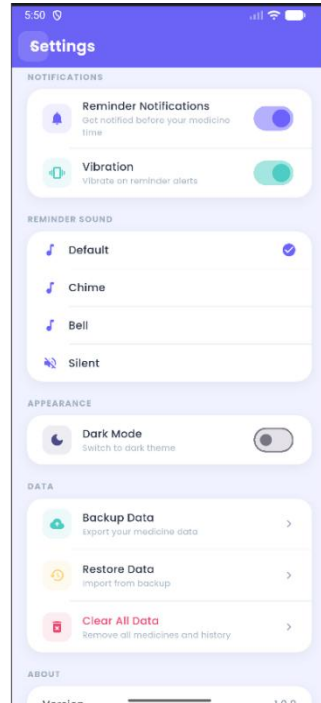
Add Medicine Screen



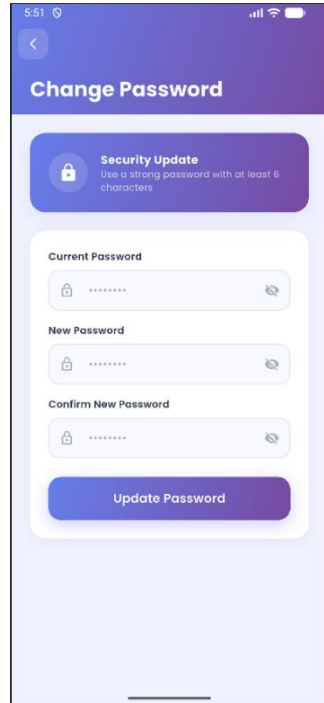
Edit Profile Screen



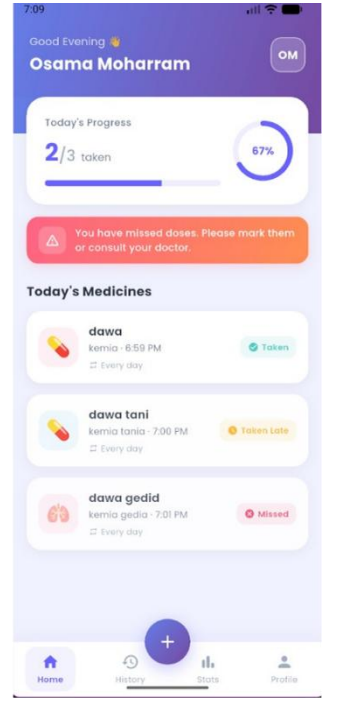
Profile Screen



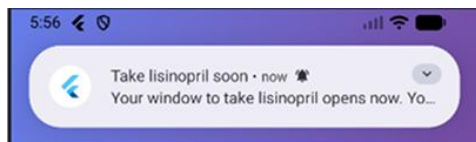
Settings Screen



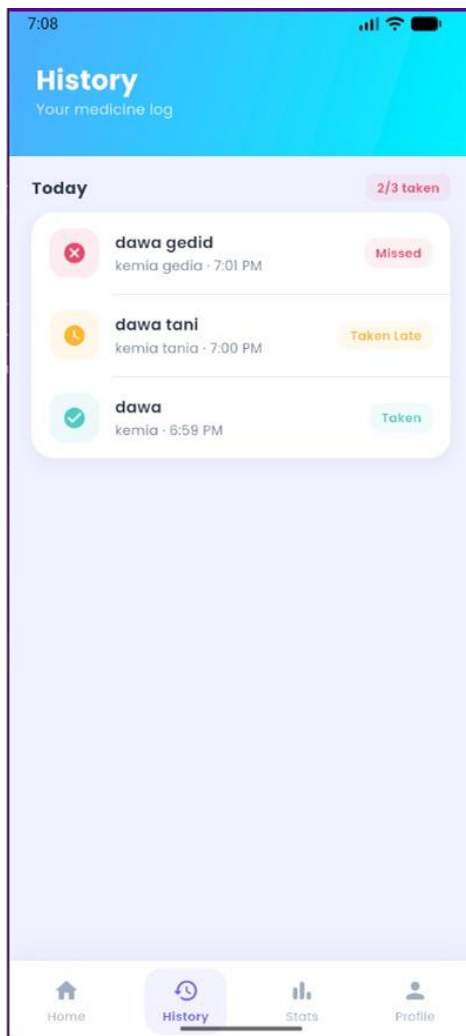
Change Password Screen



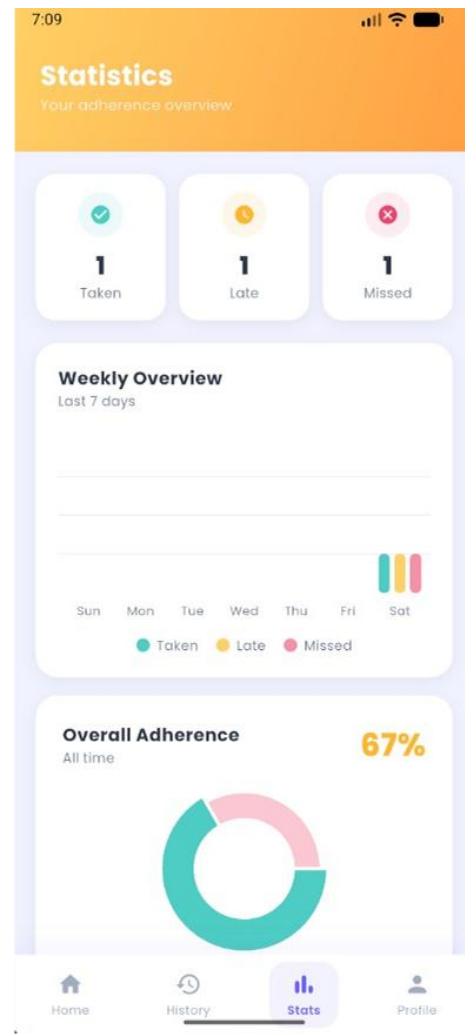
Home Screen



Notification



History Screen



Statistics Screen

7. Schedule and Budget

[Marwan Ahmed & Ali Essam]

7.1 Project Schedule

The development of the MediCare application was completed over a limited academic period. The project was divided into several phases as follows:

- Week 1–2: Requirements analysis and SRS documentation
- Week 3: UI/UX design using Figma
- Week 4–5: Application development using Flutter
- Week 6: Testing and debugging
- Week 7: Final adjustments and report completion

This schedule ensured a structured and organized development process.

7.2 Budget

The MediCare application was developed as part of an academic project and did not require any financial budget.

All tools and technologies used, including Flutter, Dart, and Figma, were available as free resources.

8. Appendices

[Marwan Ahmed & Ali Essam]

This section includes additional supporting information related to the MediCare application.

- Data Model: User, Medicine, and HistoryEntry data are stored as JSON in `shared_preferences`, namespaced per user using a base64-encoded email key to prevent data collision between accounts on the same device.
- System Notes: The application uses local storage instead of cloud-based services, ensuring that all data is stored on the user's device.
- Additional Materials: Any extra diagrams or screenshots that support the system design.

References

1. Flutter Team. (2024). *Flutter documentation*. <https://docs.flutter.dev>
2. Dart Team. (2024). *Dart programming language overview*. <https://dart.dev/guides>
3. Google Developers. (2024). *Android developers documentation: Local data storage*. <https://developer.android.com/training/data-storage>
4. Apple Inc. (2024). *iOS local data storage guidelines*. <https://developer.apple.com/documentation>
5. Flutter Community. (2024). *flutter_local_notifications package documentation*. https://pub.dev/packages/flutter_local_notifications
6. Figma Inc. (2024). *Figma design tool*. <https://www.figma.com>
7. Flutter Community. (2024). shared_preferences package documentation. https://pub.dev/packages/shared_preferences
8. Flutter Community. (2024). fl_chart package documentation. https://pub.dev/packages/fl_chart
9. Flutter Community. (2024). uuid package documentation. <https://pub.dev/packages/uuid>